DATA MINING & BUSINESS INTELLIGENCE

Laboratory Experiment Record

EXPERIMENT NO. 5

To Implement Clustering Algorithms (K-Means, Agglomerative and DBSCAN) using RapidMiner — Airline Passenger Data

Student Name	_____
Roll No. / Enrollment No.	_____
Class / Semester	_____
Date of Performance	_____
Faculty Signature	_____

1. Aim

To implement the K-Means, Agglomerative (hierarchical) and DBSCAN clustering algorithms on an airline passenger dataset using the RapidMiner (Altair AI Studio) tool, and to compare the clusters they produce.

2. Theory

2.1 Clustering

Clustering is an unsupervised data mining technique. It groups records so that records in the same cluster are similar to each other and different from records in other clusters, without using any class label. Here it is used to find natural groups of airline passengers.

2.2 The Three Algorithms

Algorithm	How it works	Needs	Limitation
K-Means	Picks k centres, assigns every record to the nearest centre, then moves the centres until they settle.	Number of clusters k	Finds round clusters only and is sensitive to outliers.
Agglomerative	Starts with every record as its own cluster and repeatedly merges the two closest clusters, building a tree (dendrogram).	A linkage type, then a cut of the tree into clusters	Slow and memory-heavy, so it needs a small sample.
DBSCAN	Groups points that are densely packed together and labels isolated points as noise.	epsilon (radius) and minimal points	Hard to tune epsilon, and struggles when density varies.

2.3 Evaluation Measures (K-Means)

- Average within-centroid distance: the average distance of records from their cluster centre. A smaller distance means tighter clusters.
- Davies-Bouldin index: compares how spread out each cluster is with how far apart the clusters are. A smaller value means better separated clusters.

RapidMiner shows both values with a negative sign (its convention, so that a larger value is better). The size of the number without the sign is the actual distance or index.

3. Tool and Dataset

Tool: RapidMiner (Altair AI Studio, Educational license).

Dataset: Airline Passenger Satisfaction (Kaggle), file train.csv, 103,904 records.

Source: kaggle.com/datasets/teejmahal20/airline-passenger-satisfaction

Clustering uses distances, so only numeric attributes are used. Six were selected:

Attribute	Description
Age	Age of the passenger.
Flight Distance	Distance of the flight.

Attribute	Description
Inflight wifi service	Passenger rating (1–5) of the wifi service.
Online boarding	Passenger rating (1–5) of online boarding.
Seat comfort	Passenger rating (1–5) of seat comfort.
Inflight entertainment	Passenger rating (1–5) of inflight entertainment.

K-Means was run on all 103,904 records. Agglomerative clustering and DBSCAN were run on a random sample of 500 records, because Agglomerative clustering needs a lot of memory and time on large data.

4. Procedure

4.1 Data Preprocessing (common to all three algorithms)

1. Create a Blank Process in RapidMiner and drag the imported dataset (Retrieve) onto the canvas.
2. Add Select Attributes (include attributes) and keep only the six numeric attributes listed above.
3. Add Replace Missing Values to fill any blank values.
4. Add Normalize (Z-transformation) so all attributes are on the same scale.

4.2 K-Means

1. Add the k-Means (Clustering) operator after Normalize, tick 'add cluster attribute', and set k = 3 and max runs = 10.
2. Add Performance (Cluster Distance Performance) and connect the cluster model and the clustered data to it.
3. Connect the outputs to the results ports and click Run.

4.3 Agglomerative Clustering

1. Add a Sample operator after Normalize with a sample size of 500 records.
2. Add Agglomerative Clustering after Sample, then Flatten Clustering to cut the tree into 3 clusters.
3. Connect the outputs to the results ports and click Run.

4.4 DBSCAN

1. Keep the same 500-record Sample and add the DBSCAN operator after it, with epsilon = 0.5 and minimal points = 5.
2. Connect the outputs to the results ports and click Run. Records that do not belong to any dense cluster are labelled Noise.

5. Implementation and Output

5.1 Data Preprocessing Pipeline

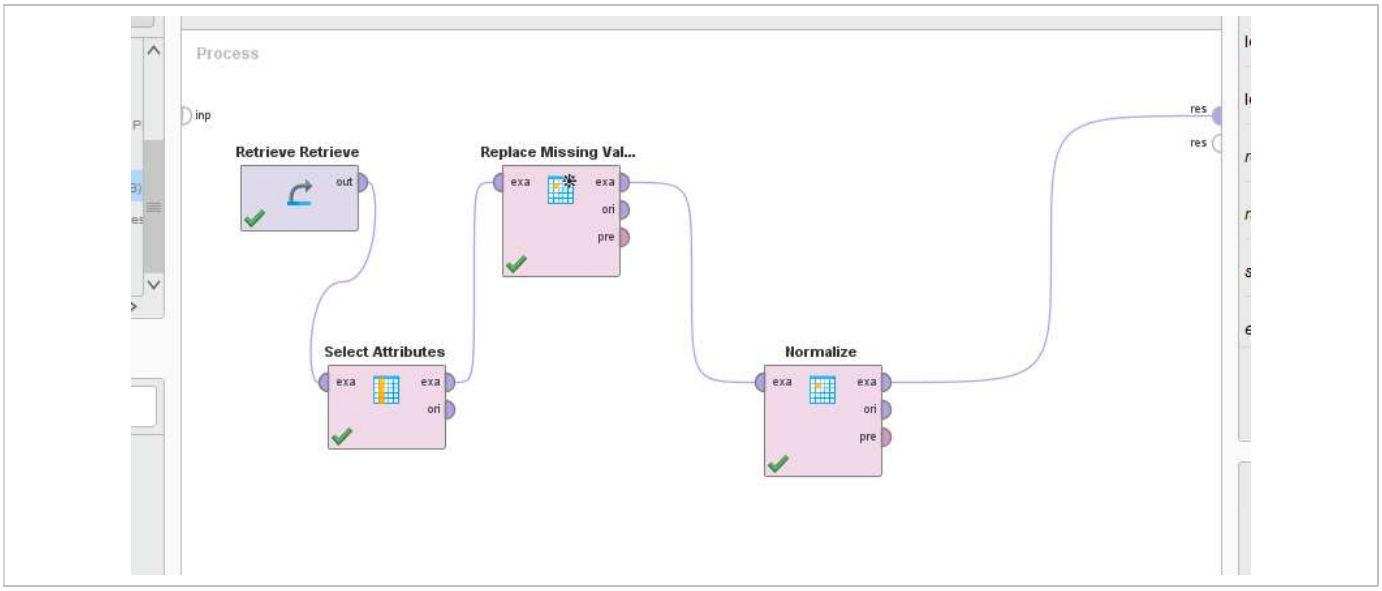


Figure 1: Preprocessing pipeline (Retrieve → Select Attributes → Replace Missing Values → Normalize)

5.2 K-Means Clustering

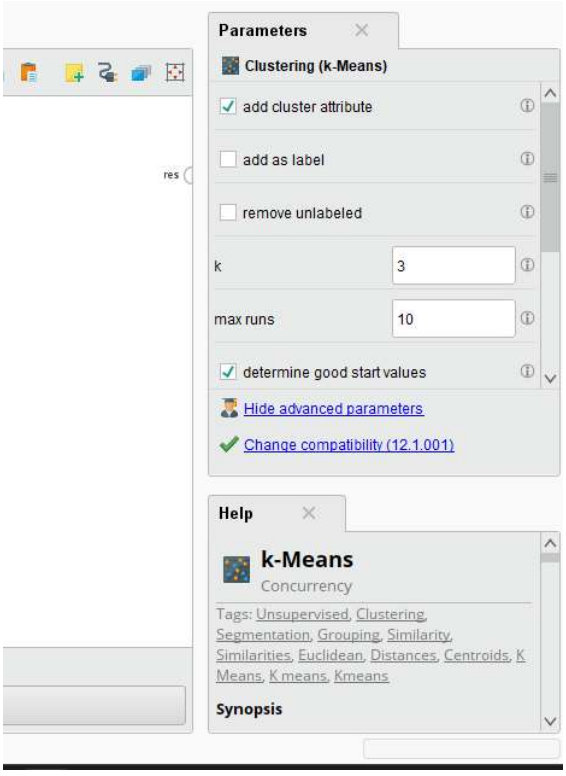


Figure 2: K-Means parameters (add cluster attribute ticked, k = 3, max runs = 10)

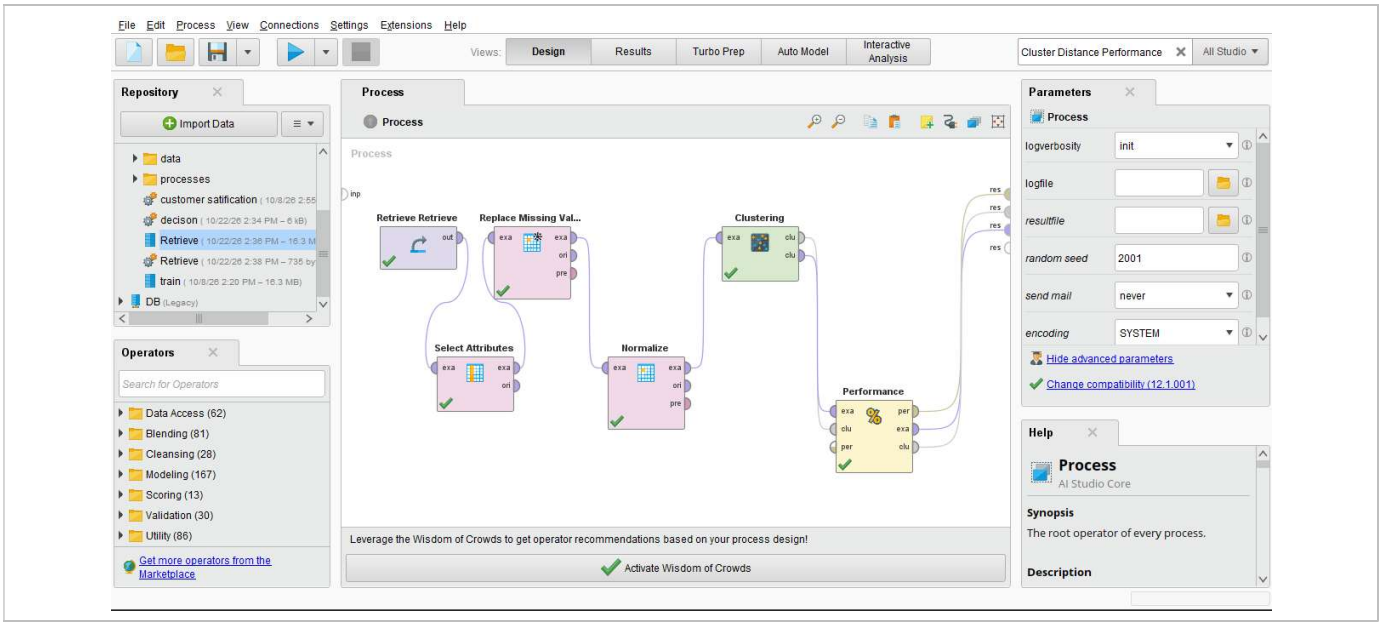
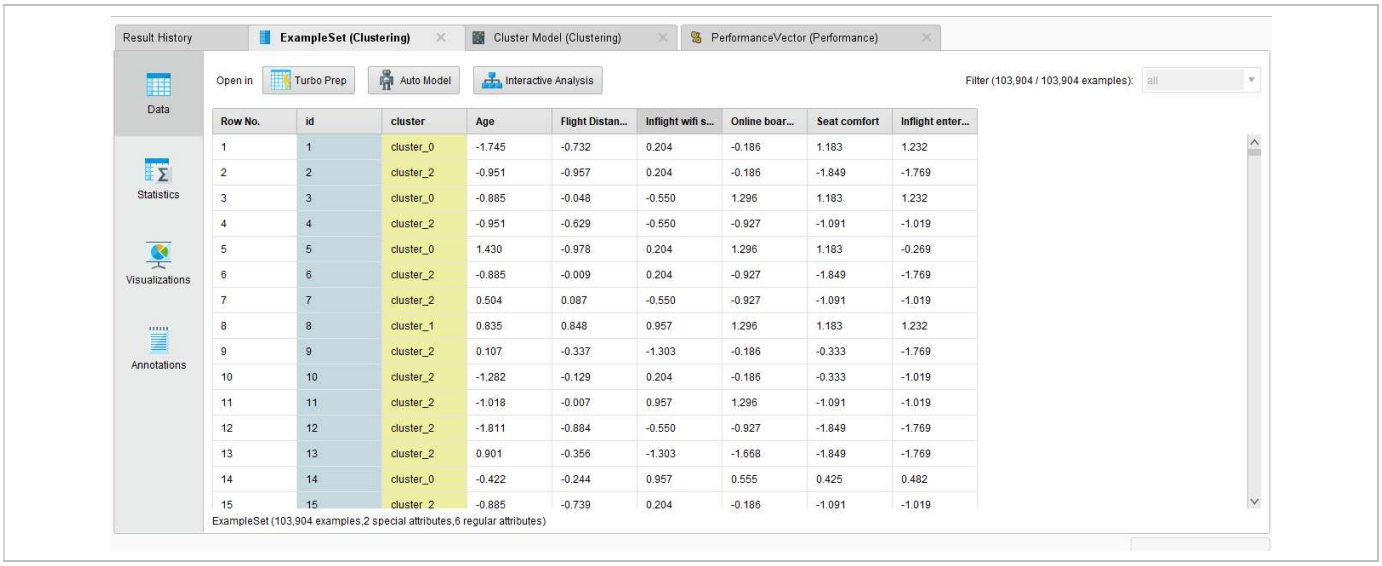


Figure 3: K-Means process in RapidMiner (Normalize → Clustering → Performance)



Row No.	id	cluster	Age	Flight Distan...	Inflight wifi s...	Online boar...	Seat comfort	Inflight enter...
1	1	cluster_0	-1.745	-0.732	0.204	-0.186	1.183	1.232
2	2	cluster_2	-0.951	-0.957	0.204	-0.186	-1.849	-1.769
3	3	cluster_0	-0.885	-0.048	-0.550	1.296	1.183	1.232
4	4	cluster_2	-0.951	-0.629	-0.550	-0.927	-1.091	-1.019
5	5	cluster_0	1.430	-0.978	0.204	1.296	1.183	-0.269
6	6	cluster_2	-0.885	-0.009	0.204	-0.927	-1.849	-1.769
7	7	cluster_2	0.504	0.087	-0.550	-0.927	-1.091	-1.019
8	8	cluster_1	0.835	0.848	0.957	1.296	1.183	1.232
9	9	cluster_2	0.107	-0.337	-1.303	-0.186	-0.333	-1.769
10	10	cluster_2	-1.282	-0.129	0.204	-0.186	-0.333	-1.019
11	11	cluster_2	-1.018	-0.007	0.957	1.296	-1.091	-1.019
12	12	cluster_2	-1.811	-0.884	-0.550	-0.927	-1.849	-1.769
13	13	cluster_2	0.901	-0.356	-1.303	-1.668	-1.849	-1.769
14	14	cluster_0	-0.422	-0.244	0.957	0.555	0.425	0.482
15	15	cluster_2	-0.885	-0.739	0.204	-0.186	-1.091	-1.019

Figure 4: Clustered data — the new 'cluster' column assigns each of the 103,904 records to a cluster

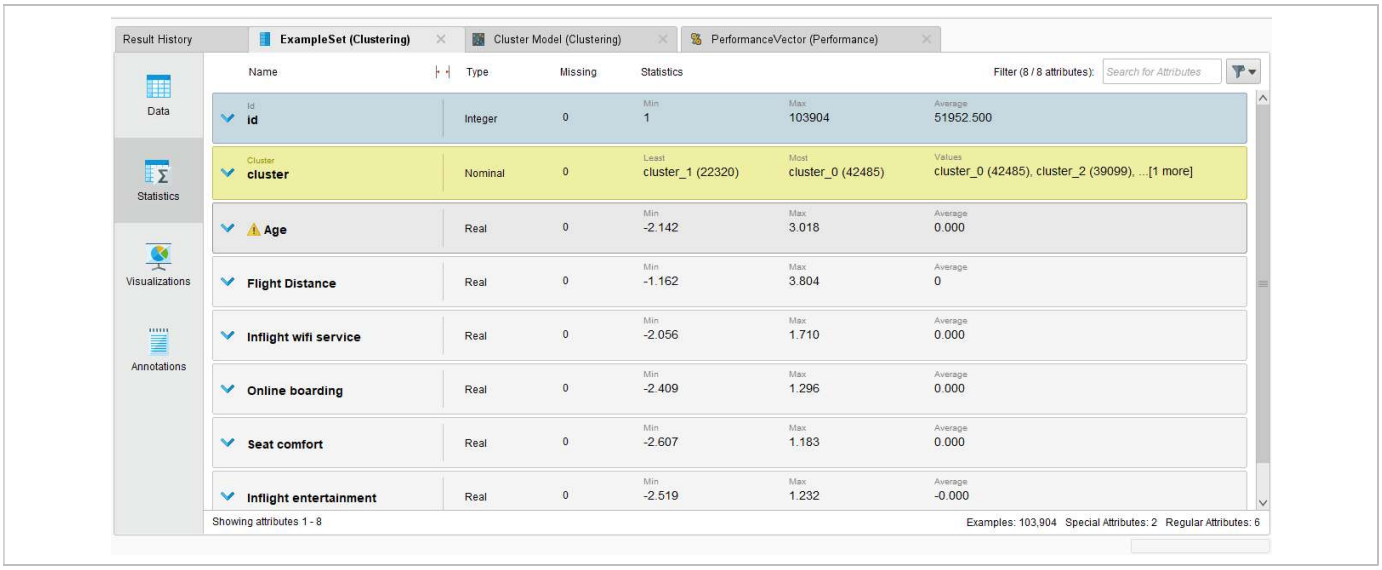


Figure 5: Statistics of the clustered data (cluster sizes shown under the cluster attribute)

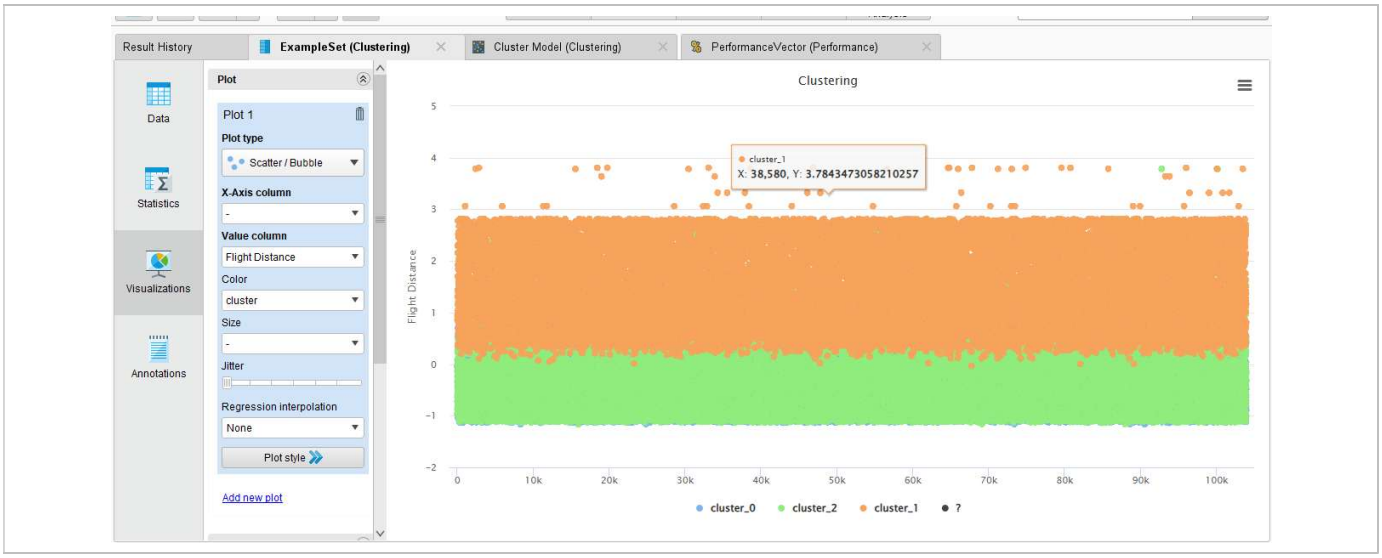


Figure 6: Scatter plot of Flight Distance, coloured by cluster

K-Means divided the 103,904 passengers into three clusters. In the scatter plot, cluster_1 (orange) holds the passengers on long-distance flights, while cluster_0 and cluster_2 hold the shorter flights.

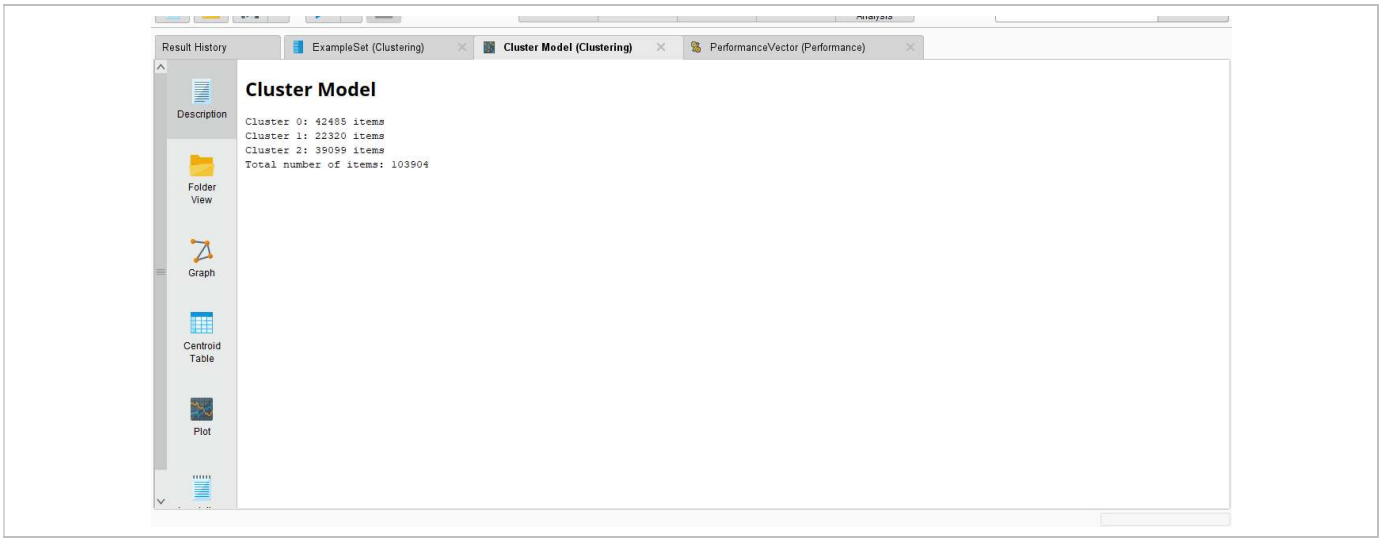
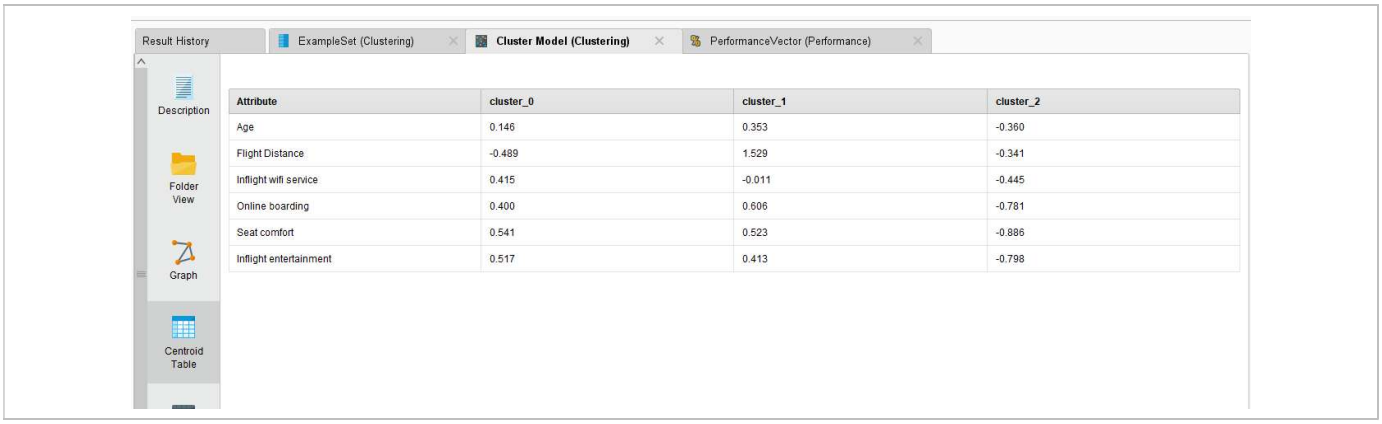


Figure 7: Cluster Model — number of records in each cluster



Attribute	cluster_0	cluster_1	cluster_2
Age	0.146	0.353	-0.360
Flight Distance	-0.489	1.529	-0.341
Inflight wifi service	0.415	-0.011	-0.445
Online boarding	0.400	0.606	-0.781
Seat comfort	0.541	0.523	-0.886
Inflight entertainment	0.517	0.413	-0.798

Figure 8: Centroid Table — the average (normalized) value of each attribute in each cluster

Cluster	Records	Profile (from the centroid table)
Cluster 0	42,485	Short-distance flights (Flight Distance −0.489) with high ratings for seat comfort, entertainment, wifi and online boarding.
Cluster 1	22,320	Long-distance flights (Flight Distance +1.529), older passengers (Age +0.353) and high ratings for online boarding, seat comfort and entertainment.
Cluster 2	39,099	Younger passengers (Age −0.360) on shorter flights with low ratings on every service (−0.445 to −0.886).

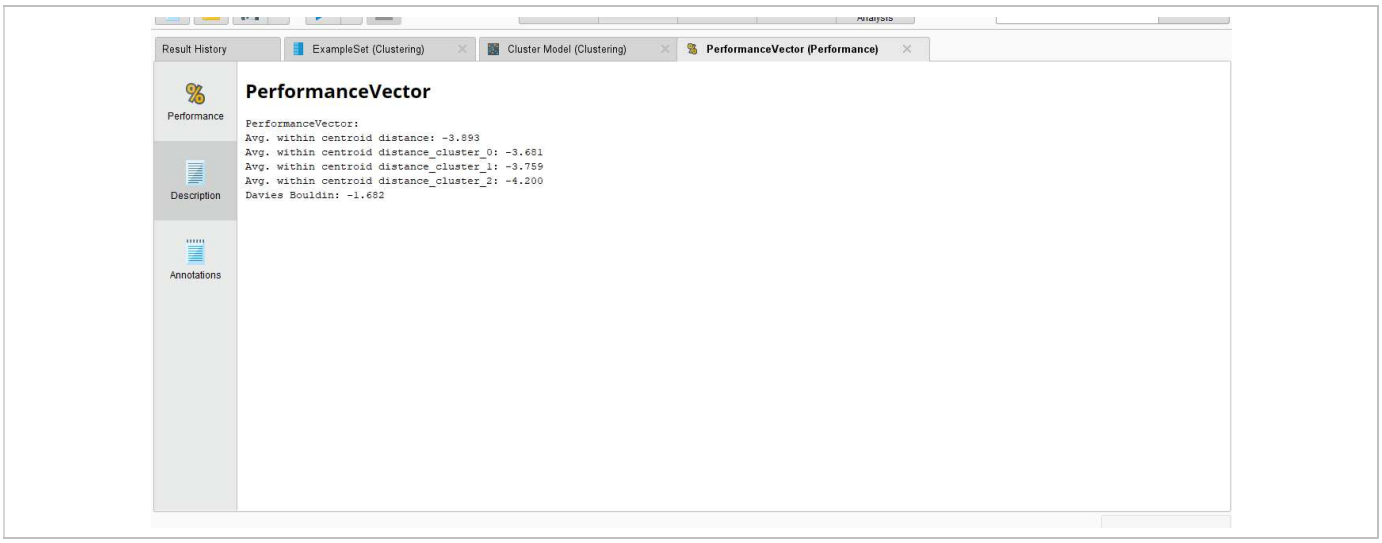


Figure 9: Performance Vector — average within-centroid distance and Davies-Bouldin index

The average within-centroid distance is 3.893 (cluster_0: 3.681, cluster_1: 3.759, cluster_2: 4.200) and the Davies-Bouldin index is 1.682. RapidMiner prints these with a minus sign; the values above are the sizes.

5.3 Agglomerative Clustering

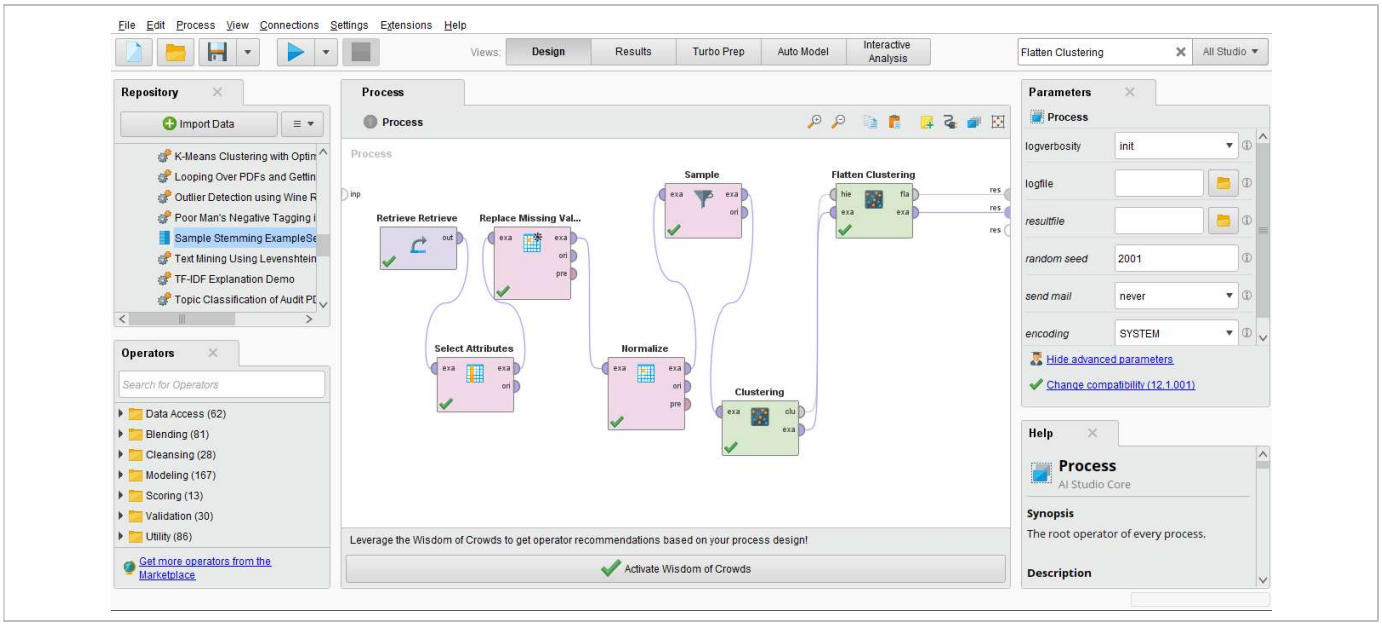


Figure 10: Agglomerative clustering process (Sample → Clustering → Flatten Clustering)

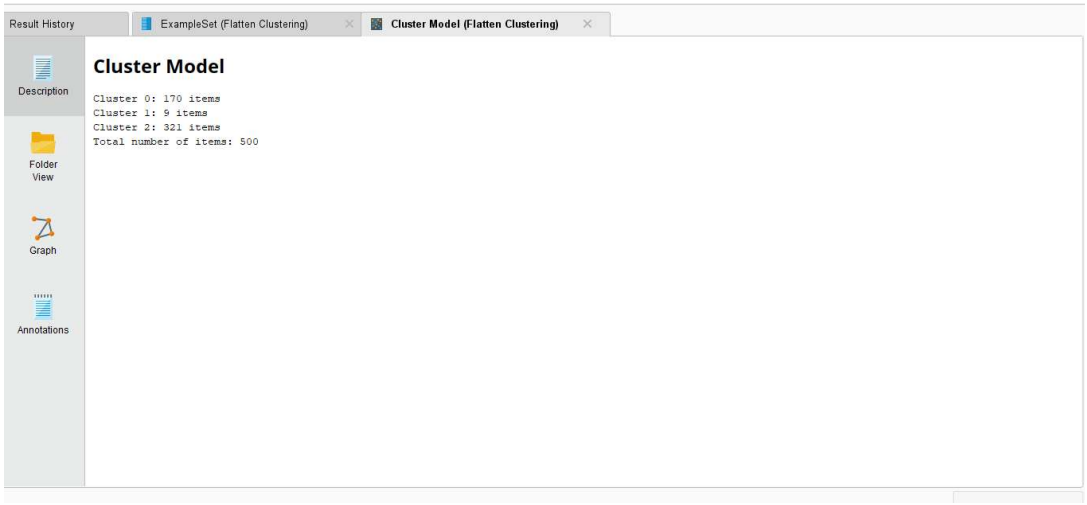


Figure 11: Cluster Model after Flatten Clustering — 500 records in three clusters

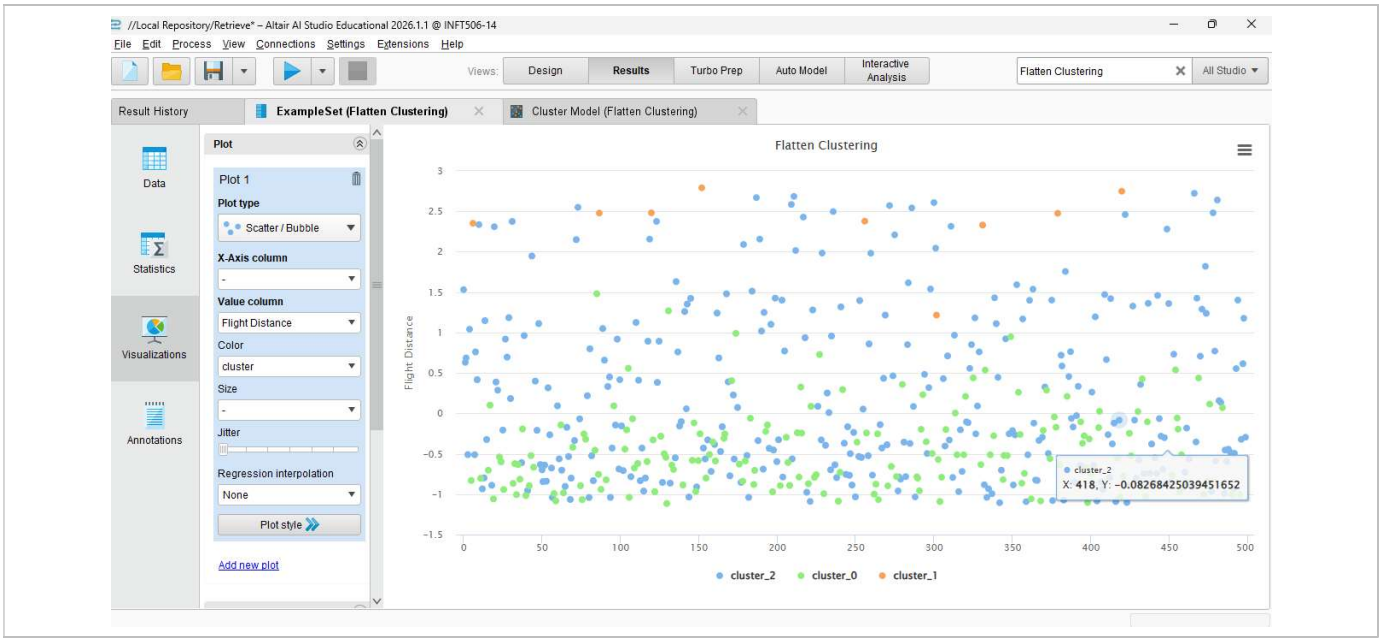


Figure 12: Scatter plot of Flight Distance, coloured by cluster (500 records)

Agglomerative clustering gave three clusters of very different sizes: cluster 0 has 170 records, cluster 1 only 9, and cluster 2 has 321. In the scatter plot, the small cluster_1 (orange) contains the few records with very high Flight Distance, so the algorithm has separated a small group of unusually long flights.

5.4 DBSCAN Clustering

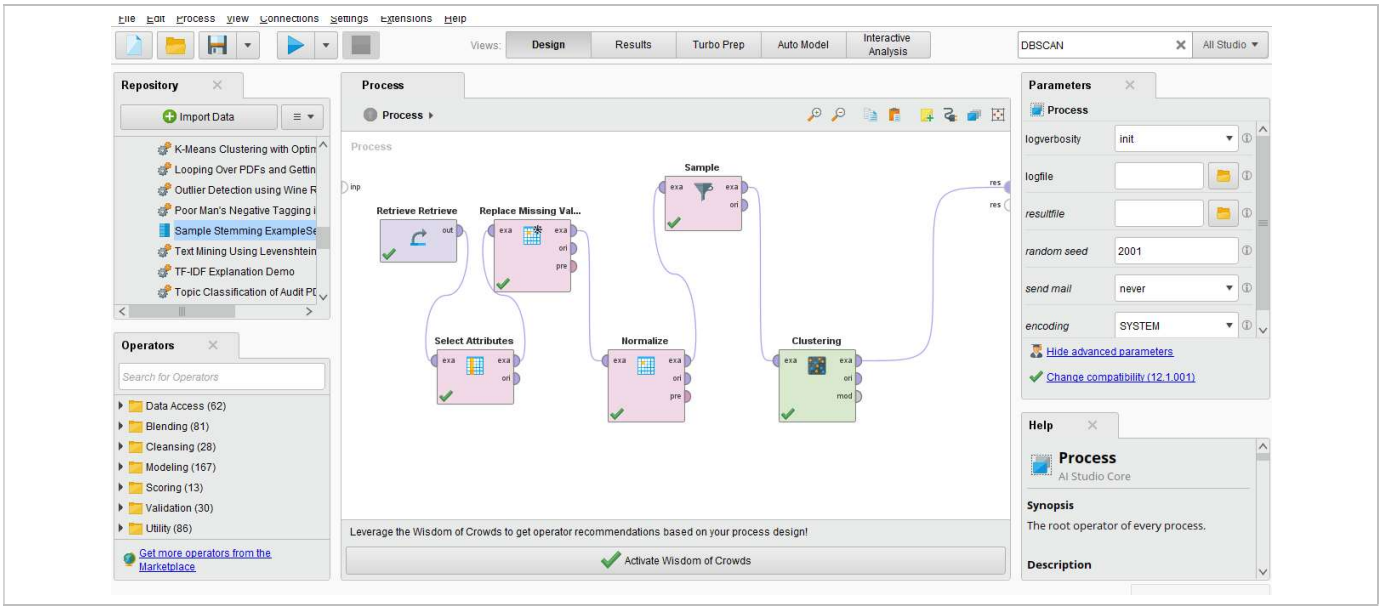


Figure 13: DBSCAN process (Sample → Clustering (DBSCAN))

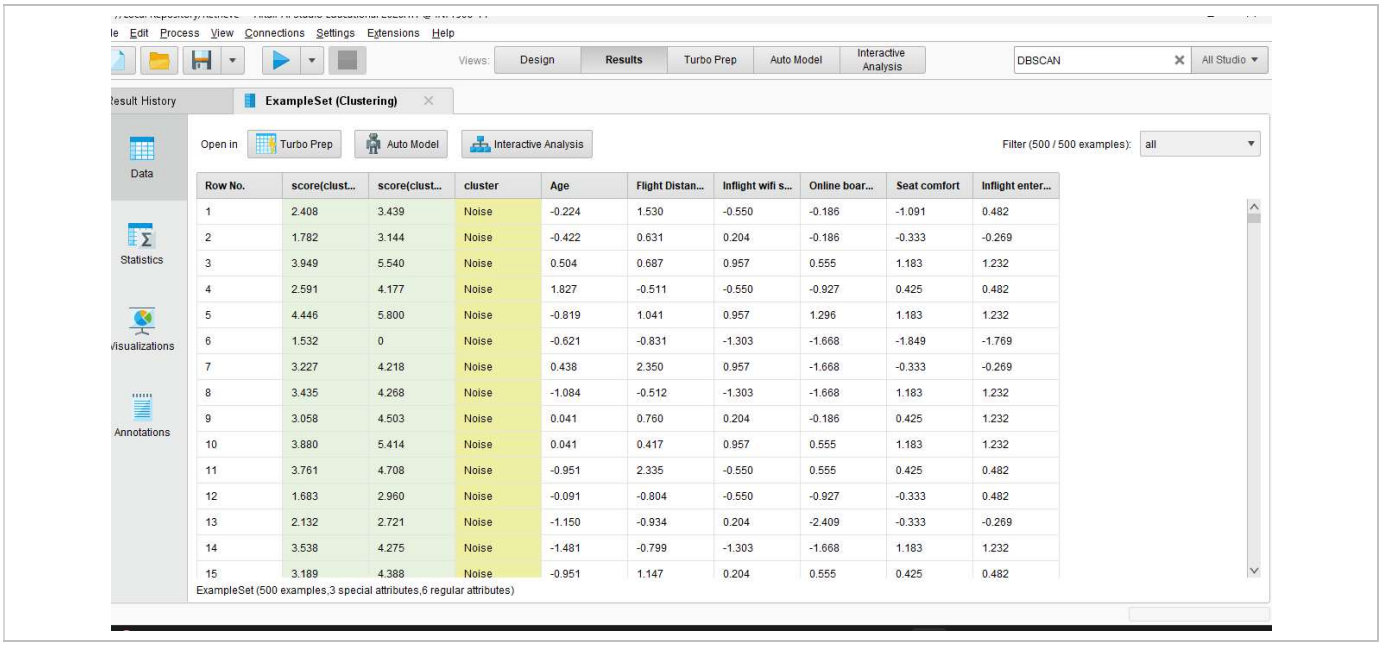


Figure 14: DBSCAN output data — cluster column and cluster scores for each record

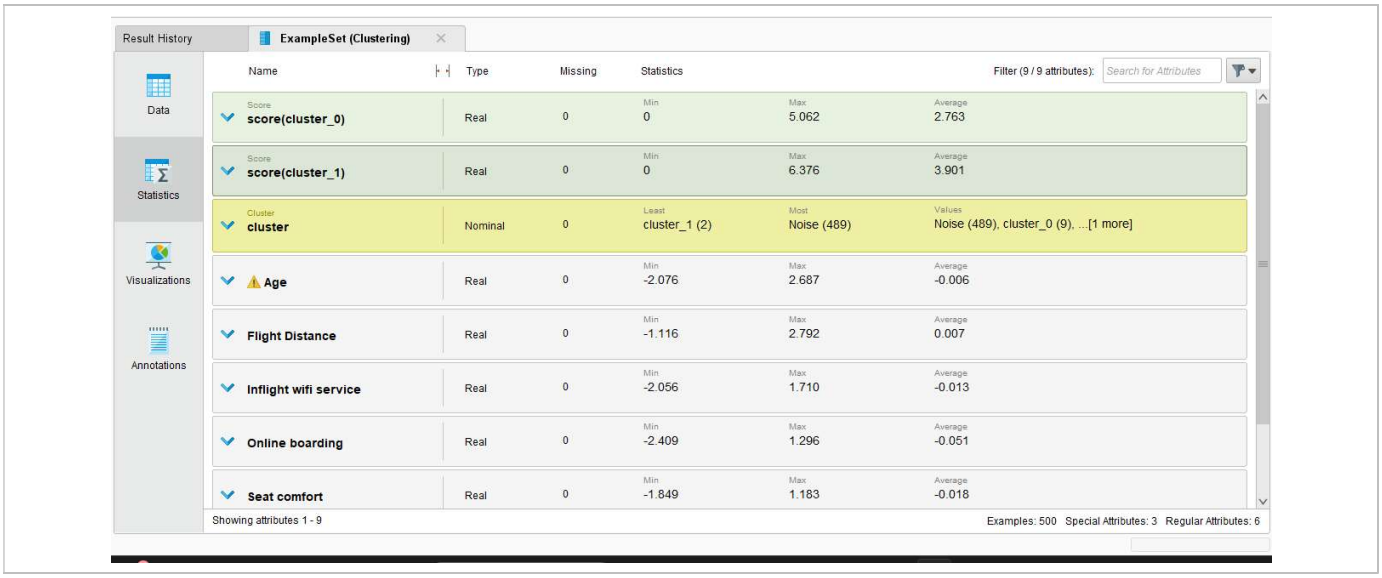


Figure 15: Statistics of the DBSCAN result — Noise (489), cluster_0 (9), cluster_1 (2)



Figure 16: Scatter plot of Age against Flight Distance, coloured by cluster

DBSCAN found two small dense clusters (cluster_0 with 9 records and cluster_1 with 2 records) and labelled the other 489 of the 500 records (97.8%) as noise. The passengers are spread out fairly evenly in this six-attribute space, so very few points have enough close neighbours to form a dense cluster with epsilon = 0.5 and minimal points = 5.

5.5 Additional Data Visualization

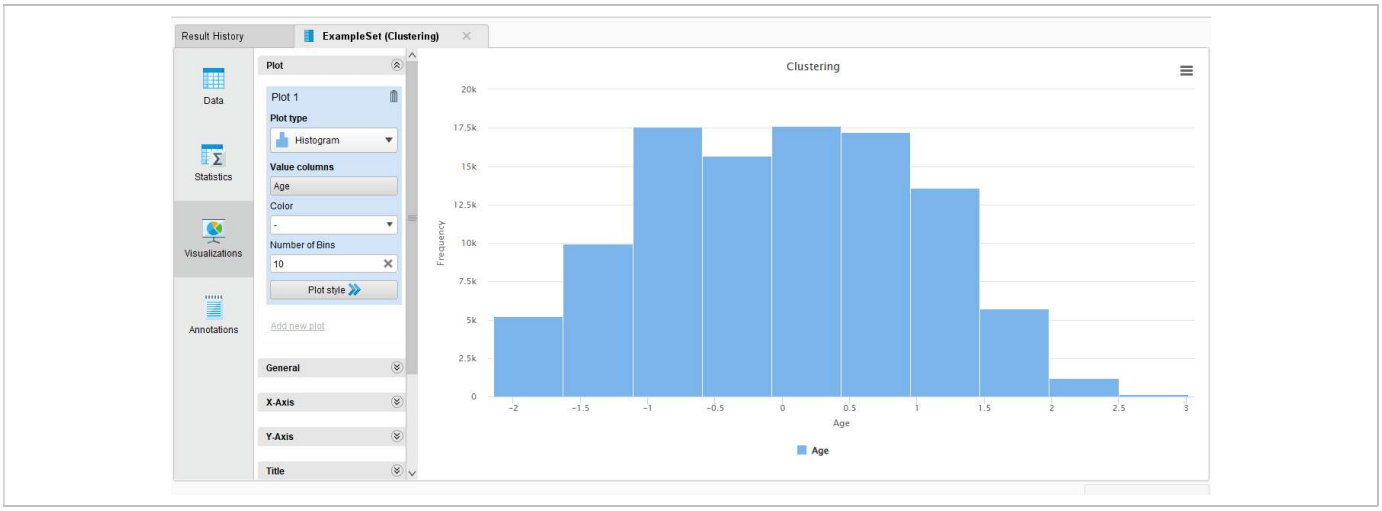


Figure 17: Histogram of the normalized Age attribute (full dataset, 10 bins)

The histogram shows that passenger ages are spread widely, with most records between about −1.5 and +1 on the normalized scale. This agrees with the DBSCAN result, as there are no sharp dense peaks in the data.

6. Result Comparison

	K-Means	Agglomerative	DBSCAN
Records used	103,904	500 (sample)	500 (sample)
Parameters	k = 3, max runs = 10	Flatten into 3 clusters	epsilon = 0.5, minimal points = 5
Clusters found	3	3	2 (plus noise)
Cluster sizes	42,485 / 22,320 / 39,099	170 / 9 / 321	9 / 2 (Noise: 489)
Noise detected	No	No	Yes — 489 records (97.8%)
Evaluation	Avg. within-centroid distance 3.893; Davies-Bouldin 1.682	Cluster sizes and scatter plot	Cluster sizes and scatter plot

7. Conclusion

K-Means, Agglomerative clustering and DBSCAN were implemented in RapidMiner on six normalized numeric attributes of the Airline Passenger Satisfaction dataset. K-Means produced three well-sized groups on the full dataset of 103,904 records: short-distance passengers with high service ratings, long-distance older passengers, and younger short-distance passengers with low ratings. Agglomerative clustering, run on a 500-record sample, produced three clusters of very unequal size and isolated a small group of very long flights. DBSCAN, with epsilon = 0.5 and minimal points = 5, found only two tiny clusters and labelled 489 of 500 records as noise.

K-Means gave the most balanced and most useful grouping for this data and is fast enough for the full dataset, but it needs k to be chosen in advance. Agglomerative clustering shows how clusters merge but is too slow for large data. DBSCAN can find noise and clusters of any shape, but it depends heavily on epsilon, and here the data has no clear dense regions, so a larger epsilon would be needed to get more useful clusters.